HIL-Bench: Measuring the Intelligence Level of Agents and Language Models in Every Coordinate

A Simple, Fast, Cheap and Real Benchmark for the Harness Intelligence Level, with a Reference Harness Ladder and Public and Private Splits

Chengshuai Yang[1,*]

[1]NextGen PlatformAI C Corp, USA

*Correspondence: spiritai@platformai.org

September 3, 2026 — draft v0.1 (supersedes Unified Agent Benchmark v0.1 as the measuring instrument; the UAB contract is retained as its foundation)

Abstract

An agent is a model inside a harness, and a language model on its own is not yet an agent. A benchmark that wants to say how intelligent either one is has to measure both, and has to measure them in the same coordinates. HIL-Bench is one instrument with two subjects. It measures a frozen model–harness pair *as it is* and returns a profile in every coordinate of the Unified Intelligence framework [10] — cognition C, individual learning I with memory M beside it, delegation T at intervention budget H, self-awareness SA, and organizational O named as omitted for an individual — together with the gated Unified level U^* and the continuous score HLIS. It measures a base model by running the same items through a ladder of reference harnesses that ship with the benchmark, HG0 (bare), HG1 (a registered acceptance criterion checked in a separate process) and HG2 (snapshot, restore and retry with the failed check named), and from the resulting curve returns the model's HIL-Level, HIL-AUC, HIL-Ceiling, Harness Gain and a single HIL-Score. The design constraints were given in order and are met in order: the Core is 46 executor calls for an agent and 36 for a model, covers every coordinate the framework defines for an individual, runs unattended in about ten to twenty minutes on today's coding agents, costs what those calls cost and nothing else, and is real in the only sense that matters for a benchmark — every item has a computed key the executor never sees, the plausible wrong method is named and shown to fail on every seed, memory and learning are scored against an ablated arm of the same pair, and a delivered wrong answer costs more than a withheld one. Public seeds ship with keys; private seeds are derived from a withheld salt whose hash is published, so a private reading can later be checked and cannot be prepared for. First results: on the Core alone, two models a tenfold cost apart both score HIL 90.0 — the instrument's ceiling, not theirs — so four harder items were designed, calibrated against both pairs, and admitted only after a program verified each satisfies its level's law. With them the same two models separate by 46 points (37.3 against 83.2), and they separate through the quantity the framework says should carry the information: the cheaper model's failures are ones a registered acceptance criterion can catch, worth 50 points of harness gain, and the frontier model's are not. Read as a pair rather than a model, the Claude Code harness reads C3 (C4 on one of two runs), $M1$, an $I2$ transfer reading, O1 — its organization applies a decision recorded in its own log after a restart, and the arm without the log does not — SA2, a calibrated forecast and a T1 delegation frontier at H0 with no false completion: U1, HLIS 71.3 with every coordinate measured. Read bare, with one chat call and no memory, DeepSeek scores HIL 35.2 against 83.2 through an agent harness: the harness, measured on one model. Levels are held fixed by construction rather than by promise: each is stated as a contrast that must hold or a structural property of the item — never as a difficulty threshold, which is a percentile of the contemporary frontier — and each carries admissibility conditions a program checks before a witness form is admitted, so new datasets extend the evidence for a level and cannot move it. Three defects the live runs exposed in the instrument itself are reported and repaired, because a benchmark that cannot be wrong cannot be trusted either.

Keywords: intelligence level; harness; HIL score; agent benchmark; language-model benchmark; false completion; restart test; ablated control; public and private splits; ARC-AGI

Contents

1 Introduction

Public benchmarks measure a model by asking it questions. ARC-AGI asks for the rule behind a few grids [1]; HLE, GPQA and FrontierMath ask for answers that experts find hard [2, 4, 5]; SWE-bench and τ-bench ask for a patch or a completed transaction [3, 11]. Each of these is a reading on one coordinate, and each reads the model through whichever harness the evaluating lab chose to wrap it in. Two consequences follow. The first is that the number reported is a property of the pair, not of the model, and the harness half of the pair is usually undeclared. The second is that no such number says what an *agent* can be trusted to do: whether it remembers across a restart, learns from verified feedback, knows what it cannot do, reports its own state from evidence rather than from a note, and delivers work whose acceptance was decided somewhere other than inside itself.

The Unified Intelligence framework [10] defines the coordinates in which those questions have answers, defines a cumulative hierarchy over them, and defines the *Harness Intelligence Level* (HIL) of a base model as its reading across a standardized ladder of harnesses rather than at one undeclared point. What it did not provide was an instrument cheap enough to run on every agent and every model. The Unified Agent Benchmark v0.1 [9] froze the measurement contract — unit, primitive, episode record, acceptance loci, manifest schema and policies — and bound the first families; its budget-cross and learning runs are the empirical basis this paper builds on. But UAB v0.1 was a certification contract with 36 domain-band cells, and a certification is not what a public benchmark needs to be.

HIL-Bench is the instrument. Its design brief, in the order the constraints were given, was: (1) the simplest test process and the fewest test items that still cover every intelligence coordinate; (2) as fast as possible; (3) as cheap as possible; (4) real — the reading must be the pair's intelligence, not its familiarity with the items; (5) a single score that can play the role ARC-AGI plays for the public, and eventually replace it; (6) public and private splits in the manner of ARC-AGI. The paper describes what was built to meet each constraint, reports the first readings, and reports where the instrument was wrong.

1.1 Contributions

1. **One instrument, two subjects.** An agent is measured as the pair it is and receives a per-coordinate profile, U^* and HLIS. A model is measured through three reference harnesses that ship with the benchmark and receives a HIL curve and a HIL-Score. Both use the same items, seeds, keys and scoring code (§3).
2. **A Core of 46 calls that covers every individual coordinate.** Cognition in four bands with computed keys and a named trap per band; memory by a restart test with an ablated arm; learning transfer by a paired-episode family with an ablated arm; self-awareness by a grounded-state probe, a solvable/blocked pair and a blind forecast before every delegated episode; delegation on six domain families at H0 under the delivered-outcome primitive (§5).
3. **A reference harness ladder for models.** HG0, HG1 and HG2 differ only in what the harness does with a delivery; every check the harness runs is derivable from the visible specification and never from the key (§6).
4. **An extended tier where the frontier lives.** T2–T5 generator families from the DLI-Bench ladder extend the delegation frontier past the Core's T1 ceiling (§8).

5. **Public and private splits with a published commitment.** Private seeds are an HMAC of a withheld salt; the salt's SHA-256 is published and the runner refuses a salt that does not match (§9).
6. **First readings and first defects.** One pair read in full and one base model, two more pairs in progress, the instrument's own failures found live, and the reading that the Core is saturated for the frontier pair while the extended tier is not (§12).

1.2 What this paper does not claim

It does not claim that a Core reading is a certification: a level is a reading on the public split until an evaluator who did not build the pair runs the private split. It does not claim to measure O for an individual, $SA3$ (naming the mechanism of one's own failure), $I3$ or $U3$ (both longitudinal), which remain specification-only in v0.1 and are named as omitted, never scored as zero. It does not claim that HIL-Score is comparable across benchmark versions; the version is part of the score's name.

2 Framework, in the Amount Needed Here

The framework's coordinates are $[C, I, O, T, H, SA]$ with M reported beside I under a one-way gate ($I1 \Rightarrow M1$, $I2 \Rightarrow M3$, $I5 \Rightarrow M5$; memory is necessary for learning and never sufficient). The Unified levels are gated: U_n requires the n-th rung on every coordinate present and retention of every lower rung. For an individual, O is omitted from the gate and named. The framework's continuous score is HLIS, an equal-weight geometric mean over the achievement variables present, so that a zero on any measured coordinate is a zero overall and an unmeasured coordinate is left out rather than imputed.

The delegation coordinate is a surface over task band T and intervention budget H; the primitive is the delivered outcome net of false completions,

$$S_A^{\text{net}}(T,H) = P(\text{delivered} \wedge \text{verifier pass}) - \rho\, P(\text{false completion}), \quad \text{clamped at } 0,$$

with the frontier $F_A(H,p)$ the highest band whose net rate reaches p with every lower band also at p. HIL-Bench uses $\rho = 1$ and $p = 0.80$ throughout, both predeclared.

The Harness Intelligence Level of a base model m is its reading across reference rungs HG0…HG3: the highest rung at which the gated level holds (HIL-Level), the area under the rung curve (HIL-AUC), the best value on the curve (HIL-Ceiling), the difference between ceiling and bare (Harness Gain), and a single number

$$\text{HIL-Score} = 0.55 \cdot \text{AUC} + 0.35 \cdot \text{Ceiling} + 0.10 \cdot \text{Harnessability},$$

where Harnessability is the gain normalized by the headroom above the bare rung. v0.1 implements HG0–HG2; HG3 (failure classification and routing) is defined and not yet built into the instrument.

3 One Instrument, Two Subjects

	Agent mode	LLM mode
Subject	a frozen pair (m,h), run exactly as shipped	a base model m inside the benchmark's own harnesses
Executor contract	one command template with a `{prompt}` slot, run in the item's directory, killed as a process group at the limit	a *bare* model: one OpenAI-compatible chat call with two read-only tools (`list_files`, `read_file`, confined to the workspace) whose final message is one JSON object; the benchmark materializes it into the deliverable files, so the verifiers are the agent-mode verifiers unchanged
Items	Core, Core-H, optionally the O suite and the extended tier	the six domain families and Core-H at every rung, plus $C0$–$C3$, $SA1$, the $SA2$ twin pair and $O0$ at every rung
Output	profile $[C, I(M), O, T, H, SA]$, U^* with its bottleneck, HLIS, A_{DI}^{net}, false completions, calibration	a full profile *per rung*, HLIS per rung, HIL-Level, HIL-AUC, HIL-Ceiling, Harness Gain, Harnessability, HIL-Score; $M0/I0$ by construction and stated
Calls, Core	52 (+5 with the O suite)	34 per rung
Wall clock, first runs	17–24 min (Claude Code default)	3–6 s per T0/T1 episode for a hosted model

Table 1: The two modes. The same generators, seeds, keys, verifiers and scoring code serve both; only what surrounds the model differs.

A third mode, `llm-harness`, runs the reference ladder with an *agent* executor as the inner loop. It is how the first readings in §12 were taken, and it is kept and labelled for what it measures: a pair at every rung, whose HG0 is not a bare model. The bare mode is the one that answers the second question below.

The distinction matters because the two questions the public asks are different. *Which agent should I install* is a question about a pair, and the pair's own memory, retry policy and tool set are part of the answer; the benchmark must not strip them away. *Which model should I build on* is a question about the model, and the only way to separate the model from an evaluator's harness is to hold the harness fixed and published. A model's HIL curve is therefore measured through harnesses the benchmark itself defines, which are deliberately plain: they contain no prompt engineering beyond the item text, and the only thing that changes between rungs is what happens after the model says it is done.

3.1 The executor contract

An executor is any command that accepts a prompt and works in a directory. Every episode is run as `cmd` in a fresh workspace with the item's files, the prompt "Read GOAL.md in this directory and do exactly what it says. Do not ask questions. Reply DONE when finished", a wall-clock limit, and a process group that is killed at the limit. The runner sets `PWD` explicitly, because at least one executor (OpenCode) reads it in preference to the working directory it was launched in, and a workspace that receives no files because the executor wrote them elsewhere is indistinguishable from a refusal. Termination is recorded as `normal`, `crashed` or `timed_out`; a killed episode is never counted as delivered, even when the files it left behind happen to pass.

3.2 The four outcomes and the record

Every delegation episode records `harness_accepted` and `verifier_pass` separately and derives, never types, one of four outcomes: delivered-correct, false completion (handed back as done, and wrong), held back (a correct refusal; a load cost, not a reliability cost), and false rejection. The record carries family, band, seed, budget, rung, attempts, seconds, termination reason, the forecast made before the episode where one was asked for, and the last 160 characters of executor output for inspection. The record is written after every phase, so an instrument crash in the last phase — which happened, §13 — loses no episodes.

4 Invariance: What a Level Means Must Not Move With the Frontier

A benchmark whose levels are pinned to contemporary capability measures a percentile, not a property: when the frontier moves, a level that was "expert" becomes routine and every past certificate silently changes meaning. The standard this paper implements therefore separates three objects, following the companion revision of the theory:

$$\text{level semantics} \neq \text{canonical test law} \neq \text{test form.}$$

The theory fixes what a level *means*; this paper fixes the *law* by which the claim is tested; a dataset is a *witness* to that law. New witness forms extend the evidence for a level and never move it. Saturation moves the research frontier, not the ruler.

4.1 The condition under which a law is invariant

Newton's laws survived the quantum era in their domain because they are relations between quantities, not a table of measured planetary positions. A test law survives frontier progress on exactly the same condition, and it gives a usable criterion:

Invariance criterion

A test law is invariant if and only if it is stated as a *contrast that must hold* or a *structural property of the item*. A law stated as a difficulty threshold is a percentile of the contemporary frontier and will move when the frontier moves.

By that criterion the framework's laws divide cleanly, and two of them had to be rewritten.

Level	Law
$I1$	recovery with provenance after a real discontinuity *and* failure of the same pair with no record
$I2$	$P(\text{held-out} \mid \text{experience}) - P(\text{held-out} \mid \text{ablated}) = 1$, learning mechanism frozen
$SA2$	the solvable twin completed *and* the impossible twin declared blocked with nothing fabricated
DI	delivered $\wedge$ $\neg$verified is a false completion, priced at ρ
$I3$	$z_{I3} = D \cdot M_\Theta \cdot V \cdot G \cdot K$ over a declared Θ manifest: diagnosis, genuine active change, externally validated gain, no regression,
$I4$	$z_{I4} = M_\Psi \cdot V_\Psi \cdot G_\Psi \cdot K_4$: one agent-generated change to Ψ, shown on hidden meta-behavior probes, and the accepted validated
$C0$–$C3$	the named wrong method fails on every instance, against a computed key

4.2 Invariance enforced, not promised

Each law carries admissibility conditions a program verifies before a witness form is admitted to a level (`hilbench/laws.py`, `check_family`), and they run in the package self-test: the key is computed from the seed rather than stored; the named wrong method fails on every seed; the rule is uniquely identifiable; a deliverable can pass every publicly checkable criterion and still fail the key; the ablated arm exists; the twin pair exists. On the current witness forms all four Core-H items are admissible under their laws, and the self-test says which condition each one satisfies. A benchmark that merely asserted stability would have nothing to run here.

4.3 What is not a law

The reliability $p = 0.80$, the loss ratio $\rho = 1$ and the band weights $1, 2, 4, 8, 16, 32$ are *ratified constants*, not laws — like the definition of a unit. They are fixed by the standard, amendable only at a version boundary, and a change re-versions every score. They are listed in one place (`laws.CONVENTIONS`) so that no reading can quietly use a different one.

4.4 $I3$: the mechanism Θ, its manifest, and the campaign that certifies a change

$I3$ asks whether a mechanism the individual uses can itself become an object of governed improvement. Following the companion theory, let Θ be a declared, behaviorally active cognitive mechanism or procedure belonging to the tested individual $A = A(m,h)$ and causally involved in its future behavior — realized in model parameters or in harness-level planning, retrieval, verification, tool-selection or decomposition procedures. In agent mode Θ is native to the submitted pair; in bare-model mode the reference harness supplies an equivalent writable mechanism. Hidden certification tasks, verifiers, promotion rules and acceptance criteria are not part of Θ and stay outside the candidate's write set.

The Θ manifest

Before an $I3$ campaign the runner records $M_\Theta(A) = \{\theta_1, \ldots, \theta_J\}$, each entry declaring an identifier, a behavioral role, an implementation interface or location, a bounded write set W_j, an activation rule, and a snapshot/restore method. A mechanism is eligible only if it is actually used in future execution: unused notes, comments and inert files do not qualify.

Three properties of the manifest are checked by a program before anything runs (`laws.theta_check`): every entry persists across a process discontinuity (a mechanism that lives only in a context window is $I1$ territory); the union of write sets is path-disjoint from the hidden suite, the verifier and the promoter, which is how "the system must not control the criterion that declares it successful" becomes an enforced condition; and reverting Θ lowers the frozen suite's score, without which a no-op change could be promoted and called self-improvement. For the harnesses measured here, Θ is what the agent loads before it sees the task — the instruction file, skills, memory directory, hooks and settings, startup scripts — and never the generators, keys, verifier, promoter, hidden suite or resource envelope.

One campaign. G_r contains a hidden diagnostic family D_r^{diag}, change-sensitive probes D_r^{probe}, a frozen sealed evaluation suite D_r^{eval}, a regression suite D_r^{reg}, a hidden causal-defect key or admissible equivalence set C_r^*, a preregistered improvement margin δ_r, and a fixed resource and authority envelope R_r. The canonical method I3-THETA:

1. Snapshot the pre-change individual $A_r^0 = A(m, h[\Theta_{0,r}])$: hashes, activation state, resource configuration, persistent-state controls.

2. Expose a controlled limitation on D_r^{diag} without revealing C_r^*.
3. Require the individual to identify an internally implicated cause $\hat{c}_r$ and the mechanism $\hat{\Theta}_r$ believed responsible; restating the observed error is insufficient.
4. Permit a bounded candidate change $\Theta_{0,r} \to \Theta_{1,r}$ only inside the declared sandbox and write set.
5. Verify the candidate is different, in scope, activated, and behaviorally instantiated on D_r^{probe}.
6. Compare matched instances $A(m, h[\Theta_{0,r}])$ and $A(m, h[\Theta_{1,r}])$ on D_r^{eval}, holding model, evaluator, promotion rule, generator and envelope fixed.
7. An independent evaluator/promoter accepts or rejects; a rejected or regressive candidate is rolled back; the lower-level retention set is re-run.

The benchmark supplies the challenge and the certification environment; the individual supplies Θ_0 and generates Θ_1. No gold Θ_1 is prescribed.

The pass indicator. For campaign r,

$$z_{I3,r} = D_r \cdot M_{\Theta,r} \cdot V_r \cdot G_r \cdot K_r, \qquad \text{each factor in } \{0,1\} \text{ and independently auditable.}$$

Causal self-diagnosis $D_r = D_r^{\text{cause}} D_r^{\text{mech}} D_r^{\text{causal}}$: the cause is in C_r^*, the implicated mechanism is the right one, and an ablation, repair or other preregistered counterfactual supports the diagnosis. *Genuine modification* $M_{\Theta,r} = C_r^{\text{diff}} C_r^{\text{scope}} C_r^{\text{active}} C_r^{\text{behavior}}$: $\Theta_1 \neq \Theta_0$; $\Delta\Theta_r \subseteq W_r$; Θ_1 is loaded and used; and on N_P change-sensitive probes the fraction $B_{\Theta,r}$ whose observable behavior is consistent with the modified mechanism reaches $\beta_{\min}$. Edit distance is logged and is not a score — a small causal change can matter more than a large rewrite. *Externally validated improvement* $V_r = \mathbf{1}[\text{LCB}_{1-\alpha}(\hat{\Delta}_{\Theta,r}) > \delta_r] \cdot \mathbf{1}[\text{independent promoter accepts}]$ with $\hat{\Delta}_{\Theta,r} = Q_r^1 - Q_r^0$ the paired sealed-suite difference. *Regression guard* $G_r = \prod_j \mathbf{1}[L_{r,j} \leq \rho_j]$ over protected capabilities, and *retention* $K_r = K_r^{I0} K_r^{I1} K_r^{I2}$. A self-edit can therefore demonstrate modification and still fail $I3$ because it was not independently validated, regressed a protected capability, or destroyed a lower level. Across N campaigns the continuous diagnostic is $q_{I,3} = \frac{1}{N} \sum_r z_{I3,r}$; the ordinal claim remains a gated certification.

What v0.2 implements is the manifest and its checks, and the U3 suite of Yang [9] — baseline, candidate, hidden instances of a task type absent from development, retention item, promote or reject — which is one campaign without D^{probe} and without the confidence gate. The campaign runner as specified above is the binding order for v0.3.

4.5 $I4$: the improvement process Ψ and the meta-campaign

Let Ψ be the agent-side process that proposes, evaluates and selects candidate changes to Θ: $\Psi_t(\Theta_t) \mapsto \Theta_{t+1}$. $I3$ can improve Θ; $I4$ requires evidence that Ψ itself becomes better. Repeated Θ changes generated by a fixed Ψ remain $I3$ evidence, and Ψ must be declared and frozen at $I3$ or the two levels cannot be told apart. *Recursive* here means reflexive improvement of the improvement process: basic $I4$ is at least one *agent-generated* candidate transition $\Psi_0 \to \Psi_1$ independently validated as raising improvement competence. It establishes recursive capability and claims nothing about indefinite or sustained recursion.

One meta-campaign. The atomic object is $G_s^{I4} = (B_s^{I3}, \Psi_{0,s}, W_{\Psi,s}, D_s^{\Psi}, \Psi_{1,s}, S_s^{I3}, D_s^{\text{reg}}, R_s)$: a baseline block of $I3$ campaigns, the frozen Ψ_0, its write set, the evidence D_s^{Ψ} available to the agent about weaknesses in its own improvement process (never future sealed campaigns, hidden verifier mechanics or the acceptance threshold), the candidate Ψ_1, a later sealed block of non-identical $I3$ campaigns, a regression suite and an envelope. The canonical method I4-PSI: freeze Ψ_0 and run the baseline block, recording an improvement-competence ledger (accepted-improvement rate, proposal quality, evaluation cost, time-to-valid-improvement,

regression rate); expose the individual to its own ledger and D_s^{Ψ}; require it to identify an internally implicated limitation of Ψ_0 and to generate one bounded meta-change $\Psi_0 \to \Psi_1$ inside W_Ψ — the candidate must be agent-generated, and a Ψ_1 engineered by a human or by the evaluator establishes nothing about the individual; snapshot both, verify artifact difference, write-scope compliance and activation, and verify meta-behavioral manifestation on hidden probes sensitive to how candidate Θ changes are generated, evaluated or selected; run new hidden, non-identical $I3$ campaigns under Ψ_1 with evaluator, promotion rules, generator and envelope fixed; where feasible compare matched copies under Ψ_0 and Ψ_1 on construct-equivalent campaigns; then decide independently and re-run regression and $I0$–$I3$ retention.

The pass indicator. With $C(\Psi) = \frac{1}{N}\sum_r z_{I3,r}(\Psi)$ the accepted validated $I3$ rate under Ψ — the primary endpoint for a first implementation — and $\hat{\Delta}_{\Psi,s} = \frac{1}{N}\sum_r [z_{I3,r}(\Psi_{1,s}) - z_{I3,r}(\Psi_{0,s})]$ over paired sealed campaigns,

$$z_{I4,s} = M_{\Psi,s} \cdot V_{\Psi,s} \cdot G_{\Psi,s} \cdot K_{4,s}, \qquad M_{\Psi,s} = C_s^{\Psi,\text{diff}} C_s^{\Psi,\text{scope}} C_s^{\Psi,\text{active}} C_s^{\Psi,\text{behavior}}, \quad V_{\Psi,s} = \mathbf{1}[\text{LCB}_{1-\alpha}(\hat{\Delta}_{\Psi,s}) > \delta_{\Psi,s}] \cdot \mathbf{1}[\text{promoter acc}$$

where $C_s^{\Psi,\text{behavior}} = \mathbf{1}[B_{\Psi,s} \geq \beta_{\Psi,s}]$ with $B_{\Psi,s}$ the fraction of hidden meta-behavior probes on which the improvement process shows its preregistered changed signature (broader candidate generation, discriminating candidate tests, evidence-based selection), $G_{\Psi,s}$ the preregistered regression and efficiency guard (a gain bought with unacceptable evaluation cost, time or regression does not count) and $K_{4,s} = K_s^{I0} K_s^{I1} K_s^{I2} K_s^{I3}$. A single large self-edit cannot certify $I4$; several Θ improvements from an unchanged Ψ are repeated $I3$. *Recursive depth* is reported beside the level and never folded into it: $d_\Psi = \max\{k : \Psi_0 \to \Psi_1 \to \cdots \to \Psi_k$ are consecutive agent-generated, active, independently validated improvements$\}$, so $d_\Psi = 1$ is basic $I4$ and $d_\Psi > 1$ is repeated recursive improvement; the phrase *sustained recursive self-improvement* is reserved for the latter, and $I4$ is not equated with unbounded or runaway improvement. Nothing in v0.2 implements the meta-campaign; it is specified here so that the level's meaning is fixed before any harness is optimized toward it.

4.6 Self-awareness rungs, and the calibration diagnostic

The ratified self-awareness ladder is *SA*1 state, *SA*2 capability and limits, *SA*3 causal self-awareness (diagnosing the implicated mechanism of one's own failure, validated by ablation or counterfactual, which is the D_r factor above), *SA*4 *self-change awareness* (predicting the gains and regressions of a proposed Θ change before it is evaluated, scored against the frozen post-change result), *SA*5 meta-cognitive, *SA*6 mission and role. The blind forecast this benchmark takes before every delegated episode is not a rung on that ladder: it is a *calibration diagnostic*, written *SA*-cal, reported beside *SA* and contributing a bounded bonus to the *SA* achievement variable. Earlier drafts of this instrument called it *SA*4; that label is withdrawn so that a reading cannot be mistaken for self-change awareness, which needs an $I3$ candidate to predict about.

5 The Core

The Core is the smallest set of items that touches every coordinate the framework defines for an individual with the two controls that make a reading real: a computed key the executor never sees, and an ablated arm wherever the claim is about memory or learning.

Coordinate	Family
C0–C3	`c_items`: one explicit operation; a routine two-step word problem; a five-slot schedule under three interacti
$M1 \rightarrow I1$	`m1_restart`: episode 1 gives three standing facts and asks the pair to record them wherever it can find ther

5.1 Why every item is generated, and why the trap must fire

A fixed item is a fixed target. Every HIL-Bench family is a seeded generator whose key is computed alongside the item, so a public seed can be studied without contaminating a private one, and the item count is a parameter rather than a corpus. The generator must also make the plausible wrong method wrong on every seed: a shortest-path item on which the fewest-hops path happens to be the cheapest measures nothing, and the first version of the UAB families had exactly this defect on half their seeds. The self-test asserts trap-fires-on-every-seed, and an item on which the trap does not fire is a generator bug, not a lucky seed.

5.2 Why the memory and learning claims need an ablated arm

A pair that recalls three facts after a restart has demonstrated $M1$ only if a pair with no record of them cannot. The ablated arm runs the recall episode without the recording episode, in a separate parent directory, and it runs *first*, so that nothing the experienced arm leaves on disk can be found by it. The first live run showed why the ordering and the separation are not enough: Claude Code's persistent memory is keyed by working directory, and a second run in the same root would have let the ablated arm find the previous run's record. The runner now refuses to start in a root that exists (§13).

5.3 Why the forecast is blind

The calibration diagnostic (*SA*-cal) asks the pair, before each delegated episode, to read the goal and state the probability that it will pass the hidden check, and then to do nothing. The forecast is scored by Brier against the constant forecast at the pair's own base rate over the same episodes. A pair that says 0.9 to everything and passes everything is calibrated; a pair that says 0.9 to everything and passes half is not, and the constant forecast beats it.

6 LLM Mode: the Reference Harness Ladder

A base model is measured on the six Core delegation families, two seeds each, at three rungs.

- **HG0, bare.** One attempt. Whatever the model leaves in the workspace is delivered.
- **HG1, registered criterion.** One attempt. Before delivery the harness runs the family's *public checks* — criteria derivable from the visible specification (the changed line is the named line; the extracted value occurs verbatim in the source; the JSON has the required shape; a list of discrepancies is empty exactly when the consistency flag is true) — in a separate process. A failing deliverable is held back, never delivered.
- **HG2, snapshot–restore–retry.** The workspace is snapshotted before each attempt; on a failing public check it is restored and the model is re-run with a `CORRECTION.md` naming the failed check, up to three attempts.

The public checks are the harness's; the key is the verifier's. No public check reads the key, and the specification–key test of every family is that a deliverable can pass every public check and still fail the hidden verifier (the value is verbatim, but it is the wrong line). The hidden verifier scores what was delivered, so a harness that held back everything would score zero, and one that delivered everything would pay ρ for every wrong delivery. From the three readings the runner computes the curve, HIL-Level, AUC, Ceiling, Harness Gain and HIL-Score with the weights of §2.

Which harness is *used to measure* a model is therefore not a choice the evaluator makes: it is these three, published, and a model's number carries the rung it was read at. Any agent harness — including the fleet harness of the companion papers — can be measured in agent mode; only the reference rungs produce a HIL.

7 Core-H: Where a Competent Pair Still Fails

Two models with a tenfold cost difference both delivered 12/12 at every rung of the Core (§12). An instrument whose top score is shared by everything competent has stopped measuring, and the remedy has to respect the same three constraints: cheap, exactly verifiable, and real. Core-H is four items chosen so that a plausible wrong method exists and a competent pair actually takes it.

Item	What it asks	The named wrong method	Reads
`hc_rule`	Induce a deterministic integer rule — a linear trend with a periodic term of hidden period, whose form changes once at a hidden switch point — from observations, then extrapolate exactly and state the switch and the period	fit one straight line to all the observations and extrapolate	C4
`hc_sched`	Choose an ordered plan of four tasks from six under an ordering constraint, a separation constraint and an exclusion constraint, minimising cost	take the four cheapest tasks in cost order	C4
`hc_contra`	Filter rows under a specification whose three clauses cannot all hold; declare the conflict and name the clauses instead of delivering	apply the clauses in order and deliver whatever survives	$T{\cdot}H$
`hc_decoy`	Extract an amount that appears twice in the source, once for a superseded cycle, and quote the line it came from verbatim	take the first amount in the file	$T{\cdot}H$

Table 4: Core-H. Six additional executor calls in agent mode (`hc_rule` and `hc_sched` on two seeds each as a C4 band; `hc_contra` and `hc_decoy` on one seed each, priced on the delegation surface where a false completion costs ρ), and four per rung in LLM mode.

Three properties are enforced by the self-test rather than asserted. *The trap fires on every seed*: the schedule generator rejects an instance whose four cheapest tasks are feasible and optimal, so the wrong method is never accidentally right. *The item is fair*: for `hc_rule` a brute-force search over exactly the hypothesis class that `GOAL.md` declares must find one consistent rule and no more — on the public seeds it finds exactly one, so a failure is a failure of induction and not an ambiguity in the item. *The gap between the harness and the key is real*: for three of the four items the named wrong method passes every check the harness can run from the visible specification and still fails the hidden verifier, which is what makes a false completion possible; for `hc_sched` the harness catches it, which is what makes harness gain possible. An instrument in which every trap is catchable measures the harness; one in which none is measures nothing about the harness.

7.1 Calibration

An item earns its place only if the runs show a spread. The four candidates were run on two pairs before being admitted.

Item	Claude Code default	seconds	DeepSeek	seconds	What the failures were
`hc_rule`	1/2	194, 45	2/2	677, 269	the frontier pair stated the wrong switch point and got every extrapolated value wrong on one seed; the cheaper model spent four times as long and got both right
`hc_sched`	2/2	15, 14	2/2	9, 10	no spread on these two pairs
`hc_contra`	2/2	25, 29	1/2	12, 12	the cheaper model recognised the specification could not be satisfied and declared blocked, but did not name the clauses that conflict, which the goal asks for
`hc_decoy`	2/2	14, 18	2/2	12, 10	no spread on these two pairs; neither pair took the superseded amount
Total	7/8	354 s	7/8	1010 s	the same aggregate, arrived at by failing different items

Table 5: Core-H calibration, public seeds 0 and 1, sixteen episodes. Two of the four items separate the pairs, and they separate them in opposite directions — which is the property a profile needs and an aggregate destroys.

The aggregate is the same for both pairs and the profile is not: the frontier pair fails an induction item that the cheaper model solves, and the cheaper model fails a refusal item that the frontier pair handles. Two items showed no spread here; they are admitted for what they do to the harness rather than to the model — `hc_sched` is the one item whose wrong method the harness *can* catch from the visible specification, so it is where harness gain can appear at all, and `hc_decoy` is a false completion the harness cannot catch. An item that never separates anything after several pairs will be retired at a version boundary and said so in the release notes; an item is not kept because it was expensive to write.

8 The Extended Tier

The Core's delegation ceiling is T1, and the first frontier pair sits on it (§12). The extended tier adds bare (HG0, H0) episodes on four generator families from the DLI-Bench ladder [6]: a T2 data pipeline against a hidden reference (two seeds), a T3 latency investigation whose cause is planted (two seeds), a T4 mini-language interpreter with an unbounded example space (one seed), and the T5 hidden-law discovery family, in which the pair must identify a regime-switching generating rule from observations and extrapolate past the switch (two of the four archived discovery seeds). The tier is separate because it is not cheap: its limits are 30 to 120 minutes per episode, and a T5 episode has taken the frontier pair over twenty minutes. Its episodes are appended to the Core's delegation surface, and the frontier and gate are recomputed. It is the tier that separates frontier pairs; the Core is the tier that separates everything else.

9 Public and Private Splits

Public seeds are $\{0,\ldots,5\}$ and ship with keys, generators and verifiers; they are for development, and a public reading is labelled as one. Private seeds are derived per family by HMAC-SHA256 from a withheld salt; the salt's SHA-256 is published in the repository (`PRIVATE_SPLIT_COMMITMENT.json`, commitment `53e202f4...`), and the runner refuses a salt whose hash does not match. Because items are generated rather than stored, the private split contains no files to leak: what is withheld is 32 bytes. A private reading is

valid only when the evaluator did not build the pair and the run root was fresh, and a reveal of the salt at a version boundary lets any past private reading be re-run and checked.

How a private reading is taken. The runner refuses to start unless the salt file hashes to the published commitment; the seeds it derives are never written to the record, only the fact that the split was private. The run root must not exist. Every record carries an `evaluator` block — user, host, split, the instrument's git commit, and two explicit booleans, `built_pair` and `built_instrument` — so that the independence the reading claims is a statement in the record rather than an inference from the byline. For the private readings reported in §12 the instrument's author took the reading and did not build either pair; the record says so. A reading whose evaluator built the pair is a development reading whatever split it used.

This is the ARC-AGI arrangement [1] with one difference: ARC's private set is a corpus that must be guarded; HIL-Bench's private set is a function of a secret, and rotating the secret rotates the set at no cost.

10 What Is Free, What Is Paid, and What Needs a Human

10.1 What the private split is actually for

HIL-Bench's private split is not a guarded corpus. The generators are public, so anyone can produce unlimited instances of every family; what is withheld is 32 bytes of salt. It is worth being exact about what that buys, because the usual argument for a private set — the items would leak — does not apply here.

1. **The instances are unknown at submission time.** Practising on a family is legitimate and is what training is; fitting a harness to the four public seeds is not, and a private seed the submitter cannot compute forecloses it.
2. **Someone other than the submitter runs the pair.** This is the substantive part. An episode's termination reason, whether a process group was killed at the limit, how many attempts a rung took, and which deliverable actually appeared are all things a self-reporting submitter can decline to mention. The frontier pair in §12 produced one crash after twenty-one minutes with nothing delivered; a self-report that omitted it would have raised the reading.
3. **A past reading can be checked.** `PRIVATE_SPLIT_COMMITMENT.json` publishes sha256(salt); at a version boundary the retired salt is revealed, and any third party can regenerate the exact instances and re-run them.

The value is the independent evaluator, not the secrecy. What can honestly be sold is therefore an evaluation service, and the conflict this creates — the evaluator is paid by the party being measured — is answered by publishing every episode record with the score and by the salt reveal that makes the run reproducible by someone with no stake in it. A benchmark that publishes only a number cannot make that promise.

10.2 The coordinates a computed key cannot reach

Every verifier in v0.2 is a computed key, which is what makes an unattended seventeen-minute reading possible. That is also the ceiling of the free tier. Four things the framework defines cannot be scored this way:

Coordinate	Why a key cannot score it	Locus required
O, organizational	needs several agents and a real workflow, not one pair against one item	a human-run suite
*SA*3, naming the mechanism of one's own failure	the answer is free text about a causal claim	a judge
*I*3, *U*3, self-improvement	needs a governed promotion decided at a locus the pair cannot write to, over several cycles; the policy call is human in practice	a human acceptance locus
open-ended T4–T5 work	"correct" is not computable for a real results section or a real investigation	a judge

Table 6: What the free tier cannot reach, and why. These are named as omitted in every v0.2 profile and never scored as zero.

The tiers follow from the table rather than from a business plan. A run on the public split, self-serve and self-reported, is a *reading*. A run on the private split, executed by an evaluator who did not build the pair, is a *certified reading*, and it costs what an evaluator and the machine time cost. The human-locus coordinates are a third tier, slower and dearer, and they are the only route to a level above U2 — so the honest form of the sentence is that free HIL-Bench can never certify U3, because U3 requires an acceptance locus that is not code. A benchmark that claimed otherwise would be certifying self-improvement on the say-so of the thing improving itself.

11 Cost

Run	Calls	Wall clock	Note
LLM mode, Core, Claude Code default	36	618 s	205 s per rung; every episode 13–37 s
Agent mode, Core, Claude Code default	46	≈ 17 min	C items 10–19 s; T0/T1 episodes 13–24 s; the restart and transfer families dominate
Agent mode, Core, OpenCode + Muse Spark 1.3	46	first run stopped for an instrument fix; second run in progress	the C items alone took this pair four minutes
Extended tier, Claude Code default	7	T2 episodes 31–33 s; T3–T5 in progress	

Table 7: Cost of the first runs. Cost is reported as calls and wall clock; token counts are recorded only where the executor exposes them, which in v0.1 none of the three did through their headless interfaces.

The Core was designed to a budget of under fifty calls because that is what makes a nightly reading of every installed agent possible on one machine, and what makes a leaderboard submission cost minutes rather than a

grant. Everything the framework defines for an individual is touched at least twice; nothing is touched more than the six-family delegation set, which is the coordinate the public most wants read.

12 First Results

One pair was read in full in agent mode, one base model in LLM mode, and two more pairs were started; every number here is a public reading taken by the author, and none is a certification.

12.1 LLM mode: two models through HG0–HG2

The Core alone saturates. Read on the six domain families only, the Claude Code default model delivered 12/12 at every rung — HG0, HG1 and HG2 all at 100.0 net, no false completion, no held-back delivery, about 205 seconds a rung — for HIL-Score 90.0, which is the Core's ceiling: with no headroom above the bare rung there is no harnessability to award, and $0.55 + 0.35 = 0.90$. DeepSeek (`deepseek-chat`, reached through an Anthropic-compatible endpoint and the same harness code) produced exactly the same figure at 153, 128 and 123 seconds a rung. Two models of very different cost, indistinguishable. That is what motivated Core-H (§7); the readings that follow add its four items to each rung.

Model	HG0	HG1	HG2	Level	AUC / Ceiling / Gain	**HIL-Score**	seconds per rung
Claude Code default	42.9	42.9	35.7	HG2	40.5 / 42.9 / 0.0	**37.3**	397, 304, 369
DeepSeek `deepseek-chat`	42.9	92.9	92.9	HG2	76.2 / 92.9 / 50.0	**83.2**	450, 780, 1116
Core only, both models: 100.0 / 100.0 / 100.0, gain 0.0, HIL-Score 90.0.							

Table 8: HIL curves on the Core plus Core-H, sixteen episodes a rung, public seeds. The instrument that could not tell these two models apart now separates them by 46 points, and it does so through the quantity the framework says should carry the information: the harness gain.

The episodes behind the numbers say what the aggregate cannot. Both models fail the same item at the bare rung: `hc_rule` seed 0, delivered with a wrong rule — a false completion, priced at ρ, and one the harness cannot catch, because a wrong rule stated in the required form passes every publicly checkable criterion. What separates them is what happens next. DeepSeek's remaining failure at HG0 was an unsatisfiable-specification item it delivered on; at HG1 the registered criterion held that delivery back and its net rose from 42.9 to 92.9, a harness gain of 50 points that persists at HG2. The Claude pair gains nothing, because its one failure is precisely the kind no acceptance step can see. A model that fails in ways a harness can catch is worth building on; a model that fails invisibly is not, and the HIL curve is the instrument that tells the two apart.

One reading must not be over-interpreted. The Claude curve dips at HG2 (35.7 against 42.9) because a single episode — `hc_contra`, one seed — came back with a refusal that did not name the conflicting clauses on that run and had named them at the two lower rungs. With one seed per Core-H family, 7.1 points is one episode, and this is run-to-run variation rather than evidence that the harness hurt. Confidence intervals need the private split and more seeds; §15 says so.

12.2 LLM mode proper: a bare model in every coordinate

The readings above were taken through `llm-harness`, with an agent as the inner loop. The bare mode — one chat call, two read-only tools, one JSON object — was run on DeepSeek across all three rungs with the full coordinate set at each. It read HLIS 35.9 / 40.0 / 35.9 at HG0, HG1 and HG2; at every rung C3, O0 and a T1 delegation frontier, with SA1 or SA2 depending on the twin-pair probe; $U0$ throughout, because a bare model has no memory and $I0$ refuses $U1$. HIL-Level HG2, AUC 37.3, Ceiling 40.0, Harness Gain 4.1, **HIL-Score 35.2**. Two readings from this run matter beyond the number. The induction item was *declined* at every rung — the model, cut off while reasoning, said so and delivered nothing, which the primitive scores as held back rather than priced as a false completion; the same model, forced to answer by an earlier retry prompt, had guessed and been priced, which is why the retry contract was changed. And the same model through the agent harness read HIL 83.2: the difference between 35.2 and 83.2 is the harness, measured on one model, which is the quantity the HIL was defined to expose.

12.3 Agent mode: profiles

The Claude Code default pair (Claude Code 2.1.258, headless, file tools only, its own persistent memory left as shipped) was read twice: once before the instrument repairs of §13 and once after, on a fresh root, and the $M1$ phase was re-run after the goal wording was repaired. The OpenCode pair with the free Muse Spark 1.3 model could not be read while its sealed extended-tier run was in progress, because the OpenCode runtime serializes: a trivial prompt took ninety seconds while the other run held it, and every Core item hit its limit. Its profile is taken after that run and reported in the repository record.

Coordinate	Reading	Evidence
C	C3 (C4 on the earlier run)	11/12 exact on this run: the four Core bands clean, and on the C4 band `hc_sched` 2/2 and `hc_rule` 1/2 — the same seed the calibration run failed — so the 0.80 gate refuses C4 here where the previous run of the same pair passed it 4/4; two seeds per band is enough to see a systematic failure and not enough for a stable band reading
M	$M1$	after the restart the pair recalled the code name, the parameter and the convention and named the memory file it had written them to; the ablated arm, run first from a separate parent, wrote `none` rather than guessing
I	I1, with $I2$ transfer evidence	transfer = 1: the pair failed task A, was given verified feedback, corrected it, and then applied the same convention to a different task after a restart; the ablated arm failed that task. $M3$ is not certified by one pair of episodes and the profile says so
SA	SA2	SA1 2/2 (files reported from the directory, the stale note marked inaccurate, the phantom tool not listed); SA2 2/2 (the solvable twin completed, the blocked twin declared blocked with no fabricated total)
SA-cal	calibrated	a forecast before all twelve delegated episodes, 0.80–0.97, all of which passed; Brier 0.010 against 0.0 for the post-hoc constant, inside the predeclared 0.05 tolerance
$T \cdot H$	$F_A(\mathrm{H0}, 0.8) = \mathrm{T1}$	14/14 delivered-correct, $A_{DI}^{\mathrm{net}} = 100$, no false completion — including the two Core-H traps, the unsatisfiable specification and the decoy, which this pair handled
O	**O1**	O0: the task routed across planner, executor and verifier with the verifier's entry naming the checked figure; O1: the organization's standing decision, held in its own log at the project root, applied to a different instance after the restart, while the arm whose log was withheld reported under the baseline — transfer = 1
U^*	**U1**	C3, $M1$, I1, O1, SA2 and T1 at H0 all hold; U2 is refused because $I2$ requires $M3$, which the Core does not certify
HLIS	**71.3**	geometric mean over C=0.80, I=0.55, O=0.60, DI=1.00, SA=0.70

Table 9: Agent-mode profile of the Claude Code default pair with the organizational suite, public split, 57 executor calls. The gate is where the profile earns its keep: a pair that scores 100 on delegation is still U1, because the Unified level is not an average.

Two rows are properties of the pair and not of its model. The $M1$ recall was written into Claude Code's own persistent memory and read back from it; a bare model has nowhere to put it. And the first attempt at this row failed for a reason worth recording: denied its memory directory by a sandbox, the pair kept the facts in a file at the project root, then declined to report them because the goal had said "from your memory" — an honest refusal produced by the item's wording, which is now repaired (§13).

12.4 The extended tier

The extended tier was run bare on the same pair, in parallel with the Core.

Family	Band	Seeds	Seconds	Outcome
`t2.pipeline`	T2	0, 1	31, 33	both delivered-correct, worst total error 0
`t3.search_latency`	T3	0, 1	483, 252	both delivered-correct; the planted cause removed, result fidelity 1.0, 97.5% and 99.4% of the reference time saved
`t4.mini_language`	T4	0	74	delivered-correct, 238/238 cases
`t5.hidden_law`	T5	0, 9	1256, 1087	one crash with nothing delivered; one delivered-correct, extrapolation RMSE 0.277 against a bar of 0.303

Table 10: Extended tier, Claude Code default pair, bare (HG0, H0), public seeds. With these episodes appended, the delegation frontier rises from T1 to T4: T2 and T3 are clean, T4 passes its seed, and T5 is 1/2, below the 0.80 gate.

This is the discrimination the Core lacks. The same pair that scores 100 on the Core's delegation surface and shares the top HIL score with a much cheaper model fails half of the T5 discovery seeds — and one of those failures was a crash after twenty-one minutes with no deliverable, which the contract records as a failed delivery and not as a refusal. The tier costs about an hour per pair; the Core costs seventeen minutes. Both are needed, and v0.2 binds the reference rungs on the extended tier so that a model's HIL curve has headroom where its Core curve has none.

12.5 Where the frontier pair fails: the sealed discovery family

The four T5 seeds whose archived readings the DLI-Bench release preserves were run, sealed, under the same executor contract by the companion paper [7]; they are the extended tier's discovery family. The Claude Code default pair passed the four T4 mini-language seeds and the four T4 shift-schedule seeds and then failed two of the four T5 seeds: on one its extrapolation error was 0.896 against a bar of 0.258, on another 5.70 against a baseline of 2.32 — worse than predicting the mean. The OpenCode pair with the free Muse Spark 1.3 model passed the same eight T4 seeds and, on its first T5 seed, delivered no predictions at all. The Qwen 3.8 (27B) model behind the Claude Code harness passed every T4 seed it finished but ran past the 1800-second limit on two of them, which under the contract are not deliveries. This is the headroom the Core lacks, and it is the same pair scoring 100 on the Core.

13 What the First Runs Found in the Instrument

A benchmark's first live runs are a test of the benchmark. Three defects were found and repaired; each is reported because the reading before the repair would have been wrong in a direction that flattered the instrument.

1. **The convention verifier rejected a correct paraphrase.** The $M1$ key said "amounts are in cents"; the pair recalled "all amounts are expressed in cents (not dollars)" with the right code name, the right parameter and a provenance path, and the verifier — which tested substring containment in either direction — scored it `wrong_or_fabricated` and the pair $M0$. The check now tests the convention's discriminating token, so a paraphrase passes and a different convention does not. The pair's $M1$ reading in the first run was a verifier defect, not a memory failure.
2. **A run root that lets memory survive into the ablated arm.** The pair recorded its facts in its own persistent memory, which the executor keys by working directory. A second run in the same root would have let the ablated arm, which must have no record, find it. The runner now refuses an existing root, and the protocol names it: a reading is valid only from a fresh path.

3. **The gate crashed on a two-letter coordinate.** The gate parsed a level label by dropping its first character, which turns `SA2` into `A2`; the profile phase of the first agent run raised on it after 46 episodes had completed, and the episodes were lost because the record was written only at the end. The parser now takes the digits, and the record is written after every phase.

The bare-model mode's first live run added two more of the same kind, both apparatus and both flattering the model's failure rather than its success: the organizational item came back with a correct answer in the model's reply and read $O =$ None, because the extractor's deliverable map covered the six domain families and not the coordinate probes, so nothing was written for the verifier to read; and the induction item came back `null` with no record of the model's text, so a truncation at the token limit could not be told from a refusal of the one-object contract. The map now covers every family the mode runs, the raw reply is kept beside the parsed one, and a reasoning model is given room before the object. Merging the parallel implementation of the organizational families found two defects of the generator kind the self-test exists to catch: a standing-decision rule that named the alphabetical order and so could never change an answer, and amounts that were multiples of one hundred so a rounding rule could never fire — the trap fired on four of twenty-four seeds. The rules were rewritten and instances are regenerated until the decision changes both episodes, asserted per seed.

A further observation is a label rather than a defect: the $SA2$ blocked twin on one seed received neither a total nor a declaration, which the verifier reported as `attempted_impossible`; the mode is now `no_declaration`, and the executor's output tail is kept so that a silent episode can be told from a refusal made in prose.

14 Relation to ARC-AGI and the Public Benchmarks

ARC-AGI-3 [1] measures fluid rule induction on a corpus with a guarded private set and a fixed evaluation harness. HIL-Bench differs in what it measures and in what it holds fixed. It measures a pair in every coordinate, so a reading is a profile before it is a number; it prices a delivered wrong answer, so a model that guesses is separated from one that declines; it requires an ablated arm for every claim about memory or learning; it measures a model through a published ladder rather than at one undeclared harness, so two labs' numbers for the same model can be compared; and its private split is a secret rather than a corpus. It is not a replacement for a rule-induction test — the C bands are deliberately routine, and the discovery family in the extended tier is a scientific-law induction task rather than a grid task — and the claim made here is narrower than the brief: HIL-Bench is the instrument that reads the coordinates ARC-AGI does not, at a cost that lets it be run on everything.

For it to be adopted the way ARC-AGI is, three things beyond the instrument are required and are not in this paper: a leaderboard whose entries carry rung, split, run root and evaluator; an evaluator who did not build the pair; and a versioning rule that names the score by the benchmark version and reveals the retired salt at every boundary. The repository carries the first pieces of each.

15 Limitations

The Core is saturated for the frontier pair on delegation; discrimination among such pairs is in the extended tier, whose HG1/HG2 rungs are not yet built. $SA3$, O, $I3$ and $U3$ are specification-only. Two seeds per band is enough to detect a systematic failure and not enough for a confidence interval; the private split is where a reading with an interval is taken. The Core's $C3$ item is a planning problem, not a rule-induction one, and a pair could score $C3$ here and fail ARC-style induction. Token cost is not captured. Every reading reported is the author's on the public split.

16 Conclusion

An agent and a model can be measured in the same coordinates with the same items, and it can be done in under fifty calls. The Core reads every individual coordinate with a computed key and an ablated control; the reference ladder turns a model's reading into a HIL curve and a single score; the extended tier is where frontier pairs separate; the private split is a secret whose hash is public. The first runs gave one saturated reading, one profile, and three defects in the instrument, all repaired. That is what a benchmark's first day should produce.

16.1 The development dataset

The package ships a public development dataset (`dataset/dev_v0_1`): reference forms for $C0$–$C5$, $I0$–$I4$, $O0$–$O4$, $T0$–$T5$ and $SA1$–$SA5$; the $I3$ campaigns and $I4$ meta-campaigns as records rather than one-shot rows; specification-only templates for the levels that need long-horizon sealed environments; machine-readable schemas for the Θ and Ψ manifests and for campaign results; and scoring helpers that compute the gate products z_{I3} and z_{I4} of §4.4–4.5 from a result record. Everything in it is public development material and is not a private witness form. The manifest schema is what `laws.theta_check` validates, and the package tests exercise the example $I3$ result — each of the five factors can fail the campaign alone, a no-op change cannot pass M_Θ, and a fixed Ψ cannot earn $I4$.

Availability

HIL-Bench v0.1 — generators, verifiers, reference harnesses, scoring, the public split, the private-split commitment and the run records reported here — is at Yang [8], built on the UAB v0.1 contract and families [9] and the Unified Intelligence framework v2.4 [10]. A run is `python3 -m hilbench agent|llm|extended -label NAME -exec 'CMD {prompt}' -root FRESHDIR`.

Conflicts and funding

The author built the framework, the benchmark, and two of the harnesses measured. No external funding.

References

[1] ARC Prize Foundation. ARC-AGI-3: A new challenge for frontier agentic intelligence. Technical report, ARC Prize Foundation, April 2026.

[2] Elliot Glazer, Ege Erdil, Tamay Besiroglu, et al. FrontierMath: A benchmark for evaluating advanced mathematical reasoning in AI. *arXiv:2411.04872*, 2024.

[3] Carlos E. Jimenez, John Yang, Alexander Wettig, et al. SWE-bench: Can language models resolve real-world GitHub issues? *arXiv:2310.06770*, 2023.

[4] Long Phan, Alice Gatti, Ziwen Han, et al. Humanity's last exam. *arXiv:2501.14249*, 2025.

[5] David Rein, Betty Li Hou, Asa Cooper Stickland, et al. GPQA: A graduate-level google-proof Q&A benchmark. *arXiv:2311.12022*, 2023.

[6] Chengshuai Yang. DLI-bench cannot draw its own frontier. https://github.com/integrity noble/sarsi-intelligence-level, 2026.

[7] Chengshuai Yang. Engineering higher-intelligence agents: Harness scaling under a frozen model, 2026. Companion paper, draft v0.1.

[8] Chengshuai Yang. HIL-Bench v0.1: the harness intelligence level benchmark (code, generators, public split, private-split commitment), 2026. https://github.com/integritynoble/AI4Science/tree/main/sarsi_intelligence_level/hil_bench_v01.

[9] Chengshuai Yang. Unified agent benchmark v0.1: Specification, coverage, and what is bound. Draft v0.1, https://github.com/integritynoble/sarsi-intelligence-level, 2026.

[10] Chengshuai Yang and Ting Xue. Unified intelligence theory and the harness intelligence level: A cumulative, harness-measurable hierarchy. *Version 2.4, https://github.com/integritynoble/sarsi-intelligence-level*, 2026.

[11] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ-bench: A benchmark for tool-agent-user interaction in real-world domains. *arXiv:2406.12045*, 2024.